AutoMate AI

Intelligent Automotive Parts Discovery Through Association Rule Mining

"Discover What Goes Together."

Technical Report — Assignment #01

AI3002 Applied Machine Learning · Fall 2026 · BS (SE)

National University of Computer & Emerging Sciences (FAST-NUCES)

Instructor: Shoaib Khan

Group members: [NAME — ROLL NUMBER], [NAME — ROLL NUMBER], [NAME — ROLL NUMBER]

Report generated: 2026-10-10

Data notice. No suitable public automotive transaction dataset could be obtained, so this project uses a deterministic SYNTHETIC educational dataset. Every table and figure in this report is computed from that dataset by the project's own code. The results demonstrate the method; they are not evidence about real customer purchasing behaviour.

1. Introduction and Problem Statement

A parts counter sells hundreds of items, and many of them are bought together: an oil change needs oil and a filter, a brake job needs pads and often fluid. A retailer that knows these combinations can suggest the missing item at the point of sale. The problem addressed here is to **discover such combinations from historical transaction baskets and turn them into explained recommendations**.

AutoMate AI implements the complete association-rule-mining pipeline required by the assignment: dataset → preprocessing → transaction representation → Apriori → frequent itemsets → association rules → recommendation engine → interactive Streamlit application. Each recommendation is justified by the rule that produced it together with its support and confidence.

The system is a rule-mining demonstrator. It is not a generative model, a diagnostic tool or a fitment database: a co-purchase rule says that two parts were often bought together, not that a part fits or is safe for a particular vehicle.

2. Application Domain and Motivation

Domain: automotive spare parts and accessories. The domain suits market-basket analysis because purchases cluster around service occasions (oil service, brake job, tune-up, detailing), which produces natural complementary sets. Intended users are parts retailers, workshop operators and automotive e-commerce businesses that want add-on suggestions, bundles or shelf-placement ideas.

Domain allocation. The assignment requires each group to confirm in the Google Classroom sheet that its domain is not taken by another group. [TEAM: confirm the automotive-parts domain in the GCR sheet and record the confirmation date here.]

3. Dataset Description

3.1 Source and provenance

We searched for a public automotive spare-parts dataset with invoice-level baskets. The closest candidate, the “Spare Parts Time Series Indian Dataset” on IEEE DataPort, requires a paid subscription, lists no invoice identifier in its schema and states no licence on its page, so it could not be used. General retail basket datasets do not contain automotive products. The project therefore uses a **synthetic dataset** produced by scripts/prepare_dataset.py with a fixed random seed (42).

The generator draws each basket from one of 12 purchase occasions (for example oil service, brake job, detailing). Each occasion has a list of parts with an inclusion probability; 25% of baskets combine two occasions and each basket receives on average 0.4 unrelated items as noise. The raw file is then deliberately degraded with duplicate lines, missing values, inconsistent item spellings and non-product lines so that preprocessing is exercised. **Consequence: every pattern found below originates from these design probabilities.**

3.2 Schema and size

Table 1. Raw file schema (data/raw_transactions.csv).

Column	Meaning	Used for mining
transaction_id	Invoice identifier (e.g. T00042)	Yes

Column	Meaning	Used for mining
item	Product name as keyed in (may be inconsistently formatted)	Yes
category	Shelf category	No — dropped
quantity	Units on the line	No — dropped (presence, not quantity, is mined)
transaction_date	Synthetic date in 2025	No — dropped
cashier_id	Synthetic cashier code	No — dropped

Table 2. Dataset summary after preprocessing.

Statistic	Value
Raw records (rows)	10,468
Raw transaction identifiers	3,012
Clean transactions (baskets)	2,998
Clean transaction–item rows	10,000
Unique items	41
Average basket size	3.34 items
Median / min / max basket size	3 / 1 / 10
Single-item baskets	339 (11.3%)
One-hot matrix density	8.1%

3.3 Preprocessing

Preprocessing is implemented in src/preprocessing.py. Each step records how many rows it removed or changed, so no data is discarded silently.

Table 3. Preprocessing audit trail (actual counts from the project dataset).

Step	Rows before	Rows after	Removed	Changed	Reason
Select mining columns	10,468	10,468	0	0	Dropped irrelevant column(s): category, quantity, transaction_date, cashier_id. No rows affected.
Remove rows with missing transaction_id	10,468	10,437	31	0	A line that cannot be assigned to a basket cannot contribute to any itemset.
Remove rows with missing item	10,437	10,386	51	0	Blank or null item names carry no information.
Normalise item names	10,386	10,386	0	282	Trimmed whitespace, collapsed repeated spaces, replaced underscores, applied Title Case so that e.g. 'ENGINE OIL' and 'engine_oil' count as one item.
Remove non-product lines	10,386	10,202	184	0	Invoice lines such as labour charges, fees and discounts are not purchasable parts.
Remove duplicate transaction-item pairs	10,202	10,000	202	0	An item counts once per basket; repeated lines must not inflate support.

In total 4.5% of raw rows were removed. 14 transaction identifiers had no product left after cleaning (for example invoices containing only a labour charge) and were dropped, leaving 2,998 baskets. Duplicate transaction–item pairs are removed before encoding, so an item counts at most once per basket and repeated lines cannot inflate support. The cleaned table is grouped by transaction_id into sets of items and then one-hot encoded into a boolean matrix of 2,998 rows × 41 columns, which is validated (non-empty, boolean, no empty rows) before mining.

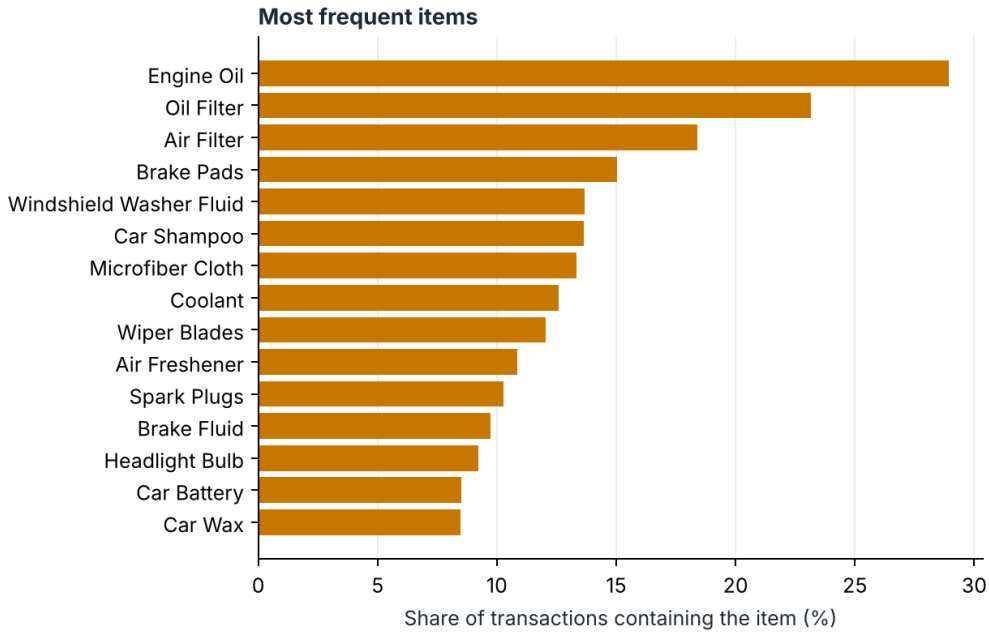


Figure 1. The 15 most frequent items. Engine Oil is the most common, appearing in 28.9% of baskets.

4. Association Rule Mining Methodology

4.1 Definitions

Let N be the number of transactions and count(X) the number of transactions that contain every item of the itemset X.

$$\text{support}(\text{X}) = \text{count}(\text{X}) \ / \ \text{N}$$

$$\text{support}(\text{A} \rightarrow \text{B}) = \text{support}(\text{A} \cup \text{B}) = \text{count}(\text{A} \cup \text{B}) \ / \ \text{N}$$

$$\text{confidence}(\text{A} \rightarrow \text{B}) = \text{support}(\text{A} \cup \text{B}) \ / \ \text{support}(\text{A}) = \text{count}(\text{A} \cup \text{B}) \ / \ \text{count}(\text{A})$$

Support is a fraction between 0 and 1 whose denominator is **all** transactions. Confidence is also a fraction between 0 and 1, but its denominator is only the transactions that contain the **antecedent** A (the left-hand side, the items already in the basket). The consequent B (right-hand side) is what the rule suggests. Confidence is an observed conditional frequency; it is not a causal effect and not a guarantee that a customer will buy B.

4.2 Workflow and implementation choices

- **Frequent itemsets.** src/apriori_engine.py contains a from-scratch, level-wise Apriori implementation (apriori_native). It was written by hand so that candidate generation and pruning are visible and can be traced step by step. The same function can delegate to the mlxtend library (backend="mlxtend"); a unit test cross-checks that both return identical itemsets and supports when mlxtend is installed.
- **Rules.** src/rule_generation.py takes only the itemsets that met the current minimum support, splits each itemset of size ≥ 2 into every non-empty antecedent/consequent pair and keeps the rules whose confidence reaches the minimum confidence.

- **Thresholds.** Minimum support and minimum confidence are interactive sliders. Changing support re-runs Apriori; changing confidence re-filters rules built from the surviving itemsets. Results are cached with keys that include the dataset file's modification time, both thresholds and the engine.
- **Lift** is shown as a supplementary column only; thresholds and ranking use support and confidence, as the assignment specifies.

4.3 Architecture

Table 4. Project modules.

Component	Responsibility
src/synthetic_data.py	Deterministic synthetic basket generator
src/data_loader.py	Loading, schema validation, synthetic fallback if the file is missing or invalid
src/preprocessing.py	Cleaning with audit trail, basket grouping, one-hot encoding, summary statistics
src/apriori_engine.py	Apriori (native + optional mlxtend), candidate generation, trace
src/rule_generation.py	Rule generation, rule search
src/recommender.py	Recommendations, explanations, what-if, threshold comparison
src/evaluation.py	Threshold experiments, example generation, manual-review join
src/visualizations.py	Plotly charts and the NetworkX association graph
src/theme.py, src/ui_html.py	Visual theme and HTML-escaped interface components
app.py	Streamlit interface (nine pages)
scripts/	prepare_dataset.py, run_experiments.py, build_report.py
tests/	59 pytest test functions with hand-verifiable fixtures

5. Apriori Explanation and Hand-Worked Example

Apriori finds all itemsets whose support reaches the minimum support by working level by level:

1. Count every single item and keep the frequent 1-itemsets (L1).
2. **Join:** combine frequent (k−1)-itemsets that share their first k−2 items into k-item candidates.
3. **Prune before counting:** discard a candidate if any of its (k−1)-subsets is not frequent.
4. **Count** the remaining candidates against the transactions and compute support.
5. **Prune after counting:** discard candidates below the minimum support. The survivors form Lk.
6. Repeat with k+1 until no candidates remain.

The pruning step relies on the **Apriori property**: if an itemset is frequent, every non-empty subset of it must also be frequent. Equivalently, any superset of an infrequent itemset is infrequent, so it need not be counted at all.

The following example uses eight invented baskets. It is illustrative only and is separate from the project dataset. The trace below was produced by the same apriori_native function that mines the real data.

Table 5. Worked example: eight illustrative baskets.

Transaction	Items
T1	Engine Oil, Oil Filter, Air Filter
T2	Engine Oil, Oil Filter
T3	Engine Oil, Oil Filter, Air Filter
T4	Air Filter, Spark Plugs
T5	Brake Pads, Brake Fluid
T6	Brake Pads, Brake Fluid, Engine Oil
T7	Air Filter, Spark Plugs, Coolant

Transaction	Items
T8	Engine Oil, Air Filter

Minimum support = 0.25, i.e. an itemset must occur in at least 2 of the 8 baskets.

Table 6. Worked example, Level 1 — single items.

Candidate	Count	Support	Outcome
{Air Filter}	5	0.625	Frequent
{Brake Fluid}	2	0.250	Frequent
{Brake Pads}	2	0.250	Frequent
{Coolant}	1	0.125	Pruned (support < 0.25)
{Engine Oil}	5	0.625	Frequent
{Oil Filter}	3	0.375	Frequent
{Spark Plugs}	2	0.250	Frequent

Table 7. Worked example, Level 2 — pairs joined from the frequent single items.

Candidate	Count	Support	Outcome
{Air Filter, Brake Fluid}	0	0.000	Pruned (support < 0.25)
{Air Filter, Brake Pads}	0	0.000	Pruned (support < 0.25)
{Air Filter, Engine Oil}	3	0.375	Frequent
{Air Filter, Oil Filter}	2	0.250	Frequent
{Air Filter, Spark Plugs}	2	0.250	Frequent
{Brake Fluid, Brake Pads}	2	0.250	Frequent
{Brake Fluid, Engine Oil}	1	0.125	Pruned (support < 0.25)
{Brake Fluid, Oil Filter}	0	0.000	Pruned (support < 0.25)
{Brake Fluid, Spark Plugs}	0	0.000	Pruned (support < 0.25)
{Brake Pads, Engine Oil}	1	0.125	Pruned (support < 0.25)
{Brake Pads, Oil Filter}	0	0.000	Pruned (support < 0.25)
{Brake Pads, Spark Plugs}	0	0.000	Pruned (support < 0.25)
{Engine Oil, Oil Filter}	3	0.375	Frequent
{Engine Oil, Spark Plugs}	0	0.000	Pruned (support < 0.25)
{Oil Filter, Spark Plugs}	0	0.000	Pruned (support < 0.25)

Table 8. Worked example, Level 3 — triples joined from the frequent pairs.

Candidate	Count	Support	Outcome
{Air Filter, Engine Oil, Oil Filter}	2	0.250	Frequent
{Air Filter, Engine Oil, Spark Plugs}	not counted	–	Pruned before counting (has an infrequent subset)
{Air Filter, Oil Filter, Spark Plugs}	not counted	–	Pruned before counting (has an infrequent subset)

Level 3 shows subset pruning at work. The candidate {Air Filter, Engine Oil, Spark Plugs} is produced by the join step, but it contains a pair that was already found to be infrequent at level 2, so by the Apriori property it cannot be frequent and is discarded without scanning the baskets. 2 of the 3 level-3 candidates are eliminated this way. Only one frequent triple remains, so no level-4 candidate can be joined and the algorithm stops.

5.1 From itemsets to rules: A → B versus B → A

The pair {Engine Oil, Oil Filter} occurs in 3 of 8 baskets, so both rules below have support 3/8 = 0.375. Their confidence differs because the denominators differ:

$$\text{confidence}(\{\text{Oil Filter}\} \rightarrow \{\text{Engine Oil}\}) = 3 \;/\; 3 = 1.00$$

$$\text{confidence}(\{\text{Engine Oil}\} \rightarrow \{\text{Oil Filter}\}) = 3 \;/\; 5 = 0.60$$

Every basket with an oil filter also has engine oil, but engine oil is also bought without a filter. A → B and B → A are therefore distinct rules: they share a numerator, yet each is divided by the support of its own antecedent.

6. Frequent Itemsets and Discovered Rules

All results in Sections 6–10 use the project dataset (2,998 baskets). At the default thresholds — minimum support 3% and minimum confidence 40% — Apriori finds **84 frequent itemsets** (37 of size 1, 34 of size 2, 12 of size 3, 1 of size 4), from which **92 rules** are generated.

Table 9. Apriori levels on the project dataset at minimum support 3%.

k	Candidates generated	Pruned before counting	Counted	Pruned after counting	Frequent
1	41	0	41	4	37
2	666	0	666	632	34
3	24	8	16	4	12
4	1	0	1	0	1

The table shows how strongly the search space is cut. Of 666 candidate pairs only 34 are frequent. At level 3 only 24 candidates are generated at all — against the 10,660 possible triples of 41 items — and 8 of them are removed by the Apriori property without being counted.

Table 10. The ten most frequent itemsets with two or more items.

Itemset	Size	Support	Transactions
{Engine Oil, Oil Filter}	2	21.7%	651
{Air Filter, Engine Oil}	2	12.9%	387
{Air Filter, Oil Filter}	2	11.7%	352
{Air Filter, Engine Oil, Oil Filter}	3	11.1%	334
{Car Shampoo, Microfiber Cloth}	2	9.2%	276
{Brake Fluid, Brake Pads}	2	8.6%	258
{Windshield Washer Fluid, Wiper Blades}	2	7.2%	216
{Drain Plug Washer, Engine Oil}	2	7.1%	214
{Drain Plug Washer, Oil Filter}	2	6.6%	198
{Brake Cleaner, Brake Pads}	2	6.6%	197

Table 11. The twelve highest-confidence rules (of 92). The full list is in reports/results/rules.csv.

Antecedent (LHS)	Consequent (RHS)	Support	Confidence
{Cabin Air Filter, Oil Filter}	{Engine Oil}	5.4%	96.4%
{Air Filter, Oil Filter}	{Engine Oil}	11.1%	94.9%
{Engine Flush, Oil Filter}	{Engine Oil}	4.2%	94.1%
{Oil Filter}	{Engine Oil}	21.7%	93.8%
{Brake Cleaner, Brake Fluid}	{Brake Pads}	3.9%	93.5%
{Air Filter, Drain Plug Washer, Oil Filter}	{Engine Oil}	3.4%	93.5%
{Air Filter, Drain Plug Washer}	{Engine Oil}	3.8%	92.7%
{Drain Plug Washer, Oil Filter}	{Engine Oil}	6.1%	92.4%
{Brake Discs, Brake Fluid}	{Brake Pads}	3.5%	90.6%
{Drain Plug Washer}	{Engine Oil}	7.1%	89.2%
{Air Filter, Drain Plug Washer, Engine Oil}	{Oil Filter}	3.4%	88.6%
{Brake Fluid}	{Brake Pads}	8.6%	88.4%

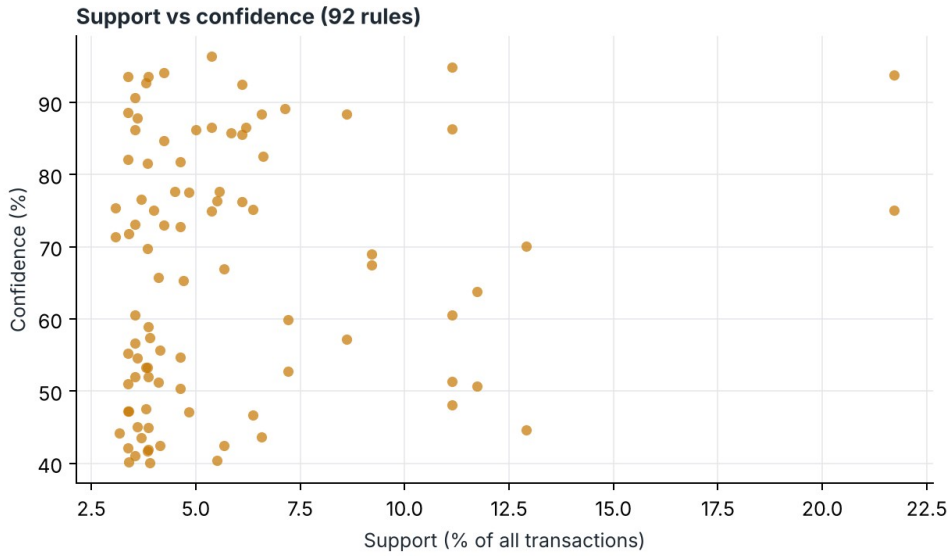


Figure 2. Support against confidence for every rule at the default thresholds. Most rules have low support; only the oil-service rules combine high support with high confidence.

Direction matters in the real results too. {Battery Terminals} → {Car Battery} and {Car Battery} → {Battery Terminals} have the same support (3.7%, 111 baskets), but confidence 76.6% versus 43.5%, because far more baskets contain a battery than contain terminals.

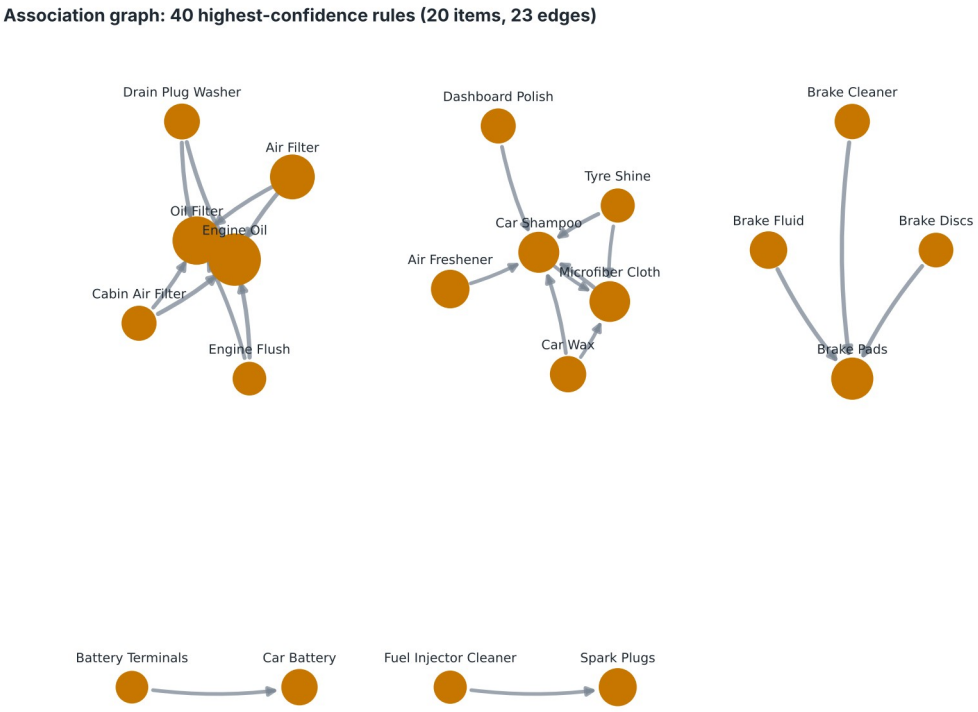


Figure 3. Association graph of the 40 highest-confidence rules. Nodes are items (size = item support). A directed edge a → b means at least one of these rules has a in its antecedent and b in its consequent; a rule with several items on one side contributes one edge per antecedent–consequent pair. Edge width increases with the highest confidence among the contributing rules.

The graph separates into clusters that mirror the service occasions (oil service, brakes, car care), with arrows converging on the main part of each cluster (Engine Oil, Brake Pads, Car Shampoo).

7. Recommendation Engine Logic

The engine (src/recommender.py, function recommend) works as follows:

1. Normalise the basket items with the same routine used in preprocessing and remove duplicates.
2. Select every rule whose antecedent is a subset of the basket. A rule with a smaller antecedent therefore also matches a larger basket.
3. Each consequent item that is not already in the basket becomes a candidate; items already selected are never recommended.
4. All matching rules that point to the same candidate are kept together as its evidence, so an item appears once however many rules support it.
5. Rank the candidates and return the top n with their supporting rules.

Ranking method. Ranked by the best supporting rule: highest confidence first, then highest support, then the more specific (larger) antecedent, then the number of supporting rules. This order is transparent and deterministic; we do not claim it is statistically optimal. When a rule has a multi-item consequent, its confidence refers to the whole consequent and is therefore a conservative figure for any single item in it.

Explanations. Every recommendation carries a sentence generated from the actual rule metrics using the template: “Customers whose transactions contained {antecedent} also purchased {consequent} in X% of the

transactions containing that antecedent. This rule has support Y% across all transactions." — where X is the rule's confidence and Y its support. The interface also lists every other supporting rule.

Safeguards. No item is recommended without a qualifying rule; items unknown to the dataset are reported rather than ignored silently; and the engine distinguishes three non-error states — empty basket, no rules at the current thresholds, and rules exist but none matches the basket.

7.1 What-if example

Basket {Engine Oil, Oil Filter, Brake Pads} yields: Brake Fluid (57%), Air Filter (51%), Brake Cleaner (44%). After removing Oil Filter, the basket {Engine Oil, Brake Pads} yields: Oil Filter (75%), Brake Fluid (57%), Air Filter (45%), Brake Cleaner (44%).

Newly recommended: Oil Filter; no longer recommended: none. Air Filter stays recommended, but its best evidence changes from {Engine Oil, Oil Filter} → {Air Filter} (51.3%) to {Engine Oil} → {Air Filter} (44.6%), because the more specific rule no longer matches. This is a comparison of rule outputs for two baskets, not a causal prediction.

8. Experiments with Support and Confidence Thresholds

The same dataset and preprocessing were mined under 4 minimum-support values × 3 minimum-confidence values (scripts/run_experiments.py). "Items with a recommendation" is the share of the catalogue for which a basket containing only that item receives at least one recommendation. The example column shows the top recommendations for the basket {Brake Pads}.

Table 12. Threshold experiment on 2,998 transactions.

Min support	Min confidence	Frequent itemsets	Largest itemset	Rules	Items with a recommendation	Example recommendations
2%	30%	130	4	200	80%	Brake Fluid (57%), Brake Cleaner (44%), Brake Discs (39%)
2%	50%	130	4	105	63%	Brake Fluid (57%)
2%	70%	130	4	68	46%	none
3%	30%	84	4	112	59%	Brake Fluid (57%), Brake Cleaner (44%), Brake Discs (39%)
3%	50%	84	4	71	49%	Brake Fluid (57%)
3%	70%	84	4	43	37%	none
5%	30%	45	3	39	39%	Brake Fluid (57%), Brake Cleaner (44%), Brake Discs (39%)
5%	50%	45	3	31	39%	Brake Fluid (57%)
5%	70%	45	3	21	27%	none
8%	30%	22	3	16	17%	Brake Fluid (57%)
8%	50%	22	3	13	17%	Brake Fluid (57%)
8%	70%	22	3	6	10%	none

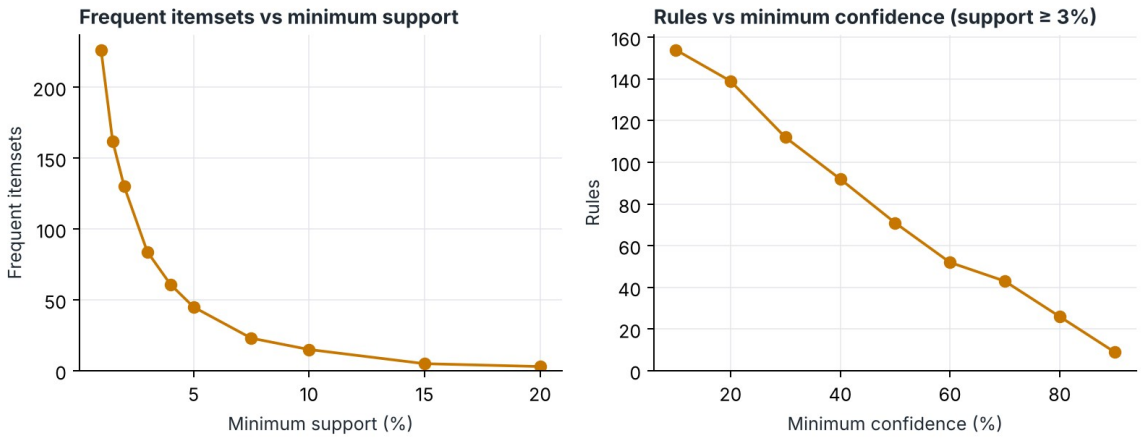


Figure 4. Left: frequent itemsets as minimum support varies (confidence fixed at 40%). Right: rules as minimum confidence varies (support fixed at 3%).

- **Support.** Raising minimum support from 2% to 8% cuts frequent itemsets from 130 to 22 and the largest itemset from 4 to 3 items. The sweep in the figure runs from 226 itemsets at 1% to 3 at 20%. Lower support admits more and larger itemsets and requires more candidates to be counted.
- **Confidence.** Confidence does not change the itemsets; it filters rules. At 2% support the rule count falls from 200 at 30% confidence to 68 at 70%.
- **Recommendation availability.** Coverage falls from 80% of items at the loosest setting to 10% at the strictest. For the example basket, the loosest setting returns Brake Fluid (57%), Brake Cleaner (44%), Brake Discs (39%), while every setting with 70% confidence returns none.
- **Trade-off.** Loose thresholds give broad coverage but include weak rules; strict thresholds keep only the strongest, mostly obvious rules and leave many baskets without any recommendation.

8.1 Recommendation comparison under two configurations

Table 13. Same dataset and basket under two threshold configurations.

Config	Min support	Min confidence	Itemsets	Rules	Recommendations for {Car Shampoo}
A	2%	30%	130	200	Microfiber Cloth (67%), Car Wax (47%), Dashboard Polish (40%), Tyre Shine (35%), Air Freshener (32%)
B	6%	70%	35	14	none

Under configuration B rules still exist (14), but none has an antecedent contained in this basket, so the application reports “no matching rule” rather than an error. The same comparison is available interactively on the Threshold Playground page.

9. Results and Interpretation

Table 14 lists recommendation examples generated by the engine at the default thresholds for 7 different baskets.

Table 14. Recommendation examples (actual engine output).

Basket	Recommended	Best supporting rule	Support	Confidence	Rules
{Engine Oil, Oil Filter}	Air Filter	{Engine Oil, Oil Filter} → {Air Filter}	11.1%	51.3%	4
{Brake Pads}	Brake Fluid	{Brake Pads} → {Brake Fluid}	8.6%	57.2%	1

Basket	Recommended	Best supporting rule	Support	Confidence	Rules
{Brake Pads}	Brake Cleaner	{Brake Pads} → {Brake Cleaner}	6.6%	43.7%	1
{Car Shampoo, Car Wax}	Microfiber Cloth	{Car Shampoo, Car Wax} → {Microfiber Cloth}	4.6%	72.8%	4
{Car Shampoo, Car Wax}	Dashboard Polish	{Car Shampoo} → {Dashboard Polish}	5.5%	40.3%	2
{Wiper Blades}	Windshield Washer Fluid	{Wiper Blades} → {Windshield Washer Fluid}	7.2%	59.8%	1
{Spark Plugs}	Air Filter	{Spark Plugs} → {Air Filter}	4.8%	47.1%	1
{Car Battery}	Battery Terminals	{Car Battery} → {Battery Terminals}	3.7%	43.5%	1
{Brake Discs, Brake Fluid}	Brake Pads	{Brake Discs, Brake Fluid} → {Brake Pads}	3.5%	90.6%	4
{Brake Discs, Brake Fluid}	Brake Cleaner	{Brake Fluid} → {Brake Cleaner}	4.1%	42.5%	1

- **{Engine Oil, Oil Filter}** — An oil-service basket is completed with the air filter; four separate rules agree, which makes this the best-supported suggestion in the set.
- **{Brake Pads}** — The engine suggests the consumables of a brake job. Both are low-cost add-ons, and fewer than six in ten pad buyers currently take either.
- **{Car Shampoo, Car Wax}** — A car-care basket is extended with the cloth needed to apply the products; the second suggestion is weaker and sits at the confidence threshold.
- **{Wiper Blades}** — A single-item basket still yields a clear complementary suggestion.
- **{Spark Plugs}** — A cross-category suggestion (ignition → filters) consistent with a tune-up.
- **{Car Battery}** — Accessory suggestion in the actionable direction, main part → add-on.
- **{Brake Discs, Brake Fluid}** — The basket contains the add-ons but not the main part; the rule points back to brake pads with very high confidence, which is correct but obvious.

Overall, the recommendations follow the service occasions built into the data, which confirms that the pipeline recovers known structure. High-confidence rules are dominated by obvious pairings that point towards the main part of a job; the commercially more interesting suggestions (main part → add-on) have moderate confidence of roughly 40–60%. Because the data is synthetic, these observations validate the method and the software, not any claim about real customers.

10. Manual Rule Evaluation

13 discovered rules were inspected by hand and assigned to the assignment's four categories. The classification and reason for each rule are stored in data/manual_rule_review.csv; support and confidence are looked up from the mined rules when the report is built, so a rule that is not actually discovered cannot appear with invented numbers.

Table 15. Manual inspection of 13 discovered rules (minimum support 3%, minimum confidence 40%).

Rule	Support	Confidence	Classification	Reason
{Oil Filter} → {Engine Oil}	21.7%	93.8%	Obvious / Trivial	An oil filter is replaced during an oil change, so nearly every oil-filter basket also holds engine oil. Any parts counter already knows this; the rule confirms the data behaves sensibly but adds no new action.
{Engine Oil, Oil Filter} → {Air Filter}	11.1%	51.3%	Useful	Roughly half of oil-service baskets add an air filter, so the other half is a concrete cross-sell opportunity at the point of sale. The rule also rests on one of the largest supports in the rule set.
{Brake Fluid} → {Brake Pads}	8.6%	88.4%	Obvious / Trivial	Brake fluid is rarely bought on its own; it accompanies pad replacement. High confidence, but it points from the add-on to the main part, which is not something a retailer needs to prompt.
{Brake Pads} → {Brake Cleaner}	6.6%	43.7%	Useful	Runs from the main part to a low-cost consumable that fewer than half of pad buyers currently take. That gap makes it an actionable add-on suggestion.

Rule	Support	Confidence	Classification	Reason
{Battery Terminals} → {Car Battery}	3.7%	76.6%	Obvious / Trivial	Terminals are an accessory to the battery, so the direction accessory -> main part is expected and has little operational value.
{Car Battery} → {Battery Terminals}	3.7%	43.5%	Useful	The actionable direction of the previous rule: same support, but clearly lower confidence because many more baskets contain a battery than contain terminals. Suggesting terminals with a battery is a plausible upsell.
{Wiper Blades} → {Windshield Washer Fluid}	7.2%	59.8%	Useful	A clear complementary pair with a majority confidence and solid support; suitable for a bundle or shelf placement.
{Headlight Bulb} → {Wiper Blades}	3.9%	41.9%	Unexpected	The two parts belong to different systems (lighting vs. wiping) and neither requires the other. The link comes only from shared 'visibility' shopping occasions, and the confidence is barely above the threshold, so it should be verified before use.
{Spark Plugs} → {Air Filter}	4.8%	47.1%	Useful	Connects ignition parts to a filter from another shelf category, consistent with a tune-up. Just under half of spark-plug baskets include it, leaving room for a prompt.
{Fuel Injector Cleaner} → {Spark Plugs}	4.0%	75.0%	Unexpected	A fuel additive predicting an ignition part is not an obvious pairing from product function alone, yet confidence is high. It hints at a 'tune-up' shopper segment; support is modest, so treat it as a hypothesis.
{Drain Plug Washer} → {Air Filter, Engine Oil, Oil Filter}	3.4%	42.1%	Weak / Uncertain	Three-item consequent with support and confidence both close to the thresholds. It is also redundant: the simpler single-consequent rules from Drain Plug Washer are stronger and easier to act on.
{Car Wax} → {Dashboard Polish}	3.4%	40.2%	Weak / Uncertain	Only just clears both thresholds, so a small change in the data or thresholds would remove it. Both items mainly co-occur with Car Shampoo, which is the more reliable signal.
{Air Freshener, Microfiber Cloth} → {Car Shampoo}	3.1%	75.4%	Weak / Uncertain	Support sits at the minimum, i.e. it rests on comparatively few baskets, and Air Freshener adds little beyond the simpler rule {Microfiber Cloth} -> {Car Shampoo}. Insufficient evidence that the extra antecedent item matters.

Category counts: Useful: 5; Obvious / Trivial: 3; Weak / Uncertain: 3; Unexpected: 2.

[TEAM REVIEW REQUIRED] These classifications are a reasoned draft. Each group member should read the rules and reasons, adjust any they disagree with in data/manual_rule_review.csv and rebuild the report. Usefulness depends on domain context, “unexpected” is judged relative to shelf category and product function, and — because the dataset is synthetic — every pattern ultimately reflects the generator's assumptions. Co-purchase association does not establish causation.

11. Limitations

- **Co-occurrence is not causation.** A rule records that items appeared in the same baskets. It does not show that buying A leads a customer to buy B, and a recommendation is not guaranteed to be accepted. In our data the associations exist only because both items belong to the same simulated purchase occasion.
- **Minimum support removes rare but potentially valuable combinations.** At 3% support an itemset needs about 90 baskets. Lowering the threshold to 1% raises the number of frequent itemsets from 84 to 226; everything in between is invisible at the default setting. Expensive, slow-moving parts (suspension, transmission) are affected most.
- **Popular items dominate.** Engine Oil appears in 28.9% of baskets and is the consequent of 11 of the 92 rules. Confidence favours frequent consequents, so the engine tends to suggest what most customers buy anyway.

- **No fitment or safety knowledge.** The system does not know vehicle make, model or year. It may suggest a part type that is sold together with another, but not the specific part that fits.
- **Sparse and single-item baskets.** 11.3% of baskets contain one item and contribute nothing to rules; at strict thresholds most items receive no recommendation (Section 8).
- **Synthetic and static data.** The rules describe a simulated past. Real purchasing changes with season, stock and price, and the system has no mechanism for detecting such drift.

12. Reproducibility, Testing and Software

The project runs locally without API keys or network access at runtime.

```
python -m venv .venv
.venv\Scripts\activate           (Windows)   |   source .venv/bin/activate
(macOS/Linux)
pip install -r requirements.txt
python scripts/prepare_dataset.py     # regenerate raw + cleaned dataset (seed 42)
python scripts/run_experiments.py     # tables and figures in reports/
python scripts/build_report.py        # this document
pytest -q
streamlit run app.py
```

Dependencies: Python 3.11+, streamlit, pandas, numpy, mlxtend, plotly, networkx, matplotlib, python-docx, pytest (see requirements.txt).

Tests. tests/ contains 59 test functions covering loading, schema validation and fallback, cleaning, basket grouping, one-hot encoding, hand-calculated support and confidence, candidate generation and pruning, rule generation and search, recommendations, exclusion of selected items, evidence combination, empty and boundary cases, what-if removal and threshold comparison.

[TEAM: run “pytest -q” on your machine and paste the summary line here, e.g. the number of tests passed.]

Appendix A. Interface Screenshots

[TEAM: run “streamlit run app.py” and insert screenshots of the working interface here. Suggested: Overview, Frequent Itemsets, Rules Explorer (with a search), Smart Basket (with the “Why this recommendation?” panel open), Visual Analytics — Association graph, Threshold Playground comparison, What-If Analysis, Apriori Learning.]

Appendix B. Team Contributions

[TEAM: list each member's contribution.]